

DAA-LAB-CHAPTER1



The screenshot shows a C++ IDE with a file named 'main.cpp'. The code is a program to check if a string is a palindrome. It includes `<stdio.h>` and `<string.h>`. The `main` function reads a string 'ada' and prints 'First palindrome: ada'. The program then checks for a palindrome by comparing characters from both ends of the string. Since 'ada' is not a palindrome, it prints 'First palindrome: \n\n' and returns 0. The output window on the right shows the program's execution, confirming that the code executed successfully and produced the expected output.

```

1 #include <stdio.h>
2 #include <string.h>
3
4 int main() {
5     int n, i, j, flag;
6     char words[1000];
7
8     printf("Enter number of words: ");
9     scanf("%d", &n);
10
11     for(i = 0; i < n; i++)
12         scanf("%s", words[i]);
13
14     for(i = 0; i < n; i++) {
15         flag = 0;
16         int len = strlen(words[i]);
17
18         for(j = 0; j < len / 2; j++) {
19             if(words[i][j] != words[i][len - j - 1]) {
20                 flag = 0;
21                 break;
22             }
23         }
24
25         if(flag) {
26             printf("First palindrome: %s\n",
27                    words[i]);
28             return 0;
29         }
30     }
31
32     printf("First palindrome: \n\n");
33     return 0;
34 }

```

Enter number of words: 3
abc car ada
First palindrome: ada

== Code Execution Successful ==



Programiz
 C Online Compiler










main.c





Run

Output

Clear

```

1 #include <stdio.h>
2
3 int exists(int a[], int n, int x) {
4     for(int i = 0; i < n; i++)
5         if(a[i] == x)
6             return 1;
7     return 0;
8 }
9
10 int main() {
11     int n, m, answer1 = 0, answer2 = 0;
12
13     printf("Enter size of nums1: ");
14     scanf("%d", &n);
15
16     int nums1[n];
17     for(int i = 0; i < n; i++)
18         scanf("%d", &nums1[i]);
19
20     printf("Enter size of nums2: ");
21     scanf("%d", &m);
22
23     int nums2[m];
24     for(int i = 0; i < m; i++)
25         scanf("%d", &nums2[i]);
26
27     for(int i = 0; i < n; i++)
28         if(exists(nums2, m, nums1[i]))
29             answer1++;
30
31     for(int i = 0; i < m; i++)
32         if(exists(nums1, n, nums2[i]))
33             answer2++;
34
35     printf("Output: [%d, %d]\n", answer1,
36           answer2);
37
38 }
```

```

Enter size of nums1: 3
2 3 2
Enter size of nums2: 2
1 2
Output: [2, 1]

=== Code Execution Successful ===
```

3. Sum of Squares of Distinct Counts

Programiz
C Online Compiler

Programiz PRO >

```
main.c
1 #include <stdio.h>
2
3 int main() {
4     int n, i, j, k, distinct, found;
5     long long sum = 0;
6
7     printf("Enter size: ");
8     scanf("%d", &n);
9
10    int nums[n];
11
12    for(i = 0; i < n; i++)
13        scanf("%d", &nums[i]);
14
15    for(i = 0; i < n; i++) {
16        int unique[100], count = 0;
17
18        for(j = i; j < n; j++) {
19            found = 0;
20
21            for(k = 0; k < count; k++)
22                if(unique[k] == nums[j])
23                    found = 1;
24
25            if(!found)
26                unique[count++] = nums[j];
27
28            distinct = count;
29            sum += distinct * distinct;
30        }
31    }
32
33    printf("Output: %lld\n", sum);
34
35    return 0;
36 }
```

Output

Enter size: 3
1 2 1
Output: 15

=== Code Execution Successful ===

4. Equal Elements and Product Divisible by K

Programiz
C Online Compiler

Programiz PRO >

```
main.c
1 #include <stdio.h>
2
3 int main() {
4     int n, k, count = 0;
5
6     printf("Enter size: ");
7     scanf("%d", &n);
8
9     int nums[n];
10
11    for(int i = 0; i < n; i++)
12        scanf("%d", &nums[i]);
13
14    printf("Enter k: ");
15    scanf("%d", &k);
16
17    for(int i = 0; i < n; i++) {
18        for(int j = i + 1; j < n; j++) {
19            if(nums[i] == nums[j] && (i * j) %
20                k == 0)
21                count++;
22        }
23    }
24
25    printf("Number of valid pairs: %d\n", count);
26
27    return 0;
28 }
```

Output

Enter size: 7
3 1 2 2 2 1 3
Enter k: 2
Number of valid pairs: 4

=== Code Execution Successful ===

5. Find Maximum Element

Programiz

C Online Compiler

Programiz PRO

main.c

Run

Output

Clear

```

1 #include <stdio.h>
2
3 int main() {
4     int n, max;
5
6     printf("Enter size: ");
7     scanf("%d", &n);
8
9     int a[n];
10
11     for(int i = 0; i < n; i++)
12         scanf("%d", &a[i]);
13
14     max = a[0];
15
16     for(int i = 1; i < n; i++)
17         if(a[i] > max)
18             max = a[i];
19
20     printf("Maximum element: %d\n", max);
21
22     return 0;
23 }

```

```

Enter size: 4
1 2 3 4
Maximum element: 4

=== Code Execution Successful ===

```

6. Sort and Find Maximum

Using Merge Sort

Programiz

C Online Compiler

Programiz PRO

main.c

Run

Output

Clear

```

1 #include <stdio.h>
2
3 void merge(int a[], int l, int m, int r) {
4     int temp[100], i = l, j = m + 1, k = 0;
5
6     while(i <= m && j <= r) {
7         if(a[i] < a[j])
8             temp[k++] = a[i++];
9         else
10            temp[k++] = a[j++];
11    }
12
13    while(i <= m)
14        temp[k++] = a[i++];
15
16    while(j <= r)
17        temp[k++] = a[j++];
18
19    for(i = l, k = 0; i <= r; i++)
20        a[i] = temp[k++];
21 }
22
23 void mergeSort(int a[], int l, int r) {
24     if(l < r) {
25         int m = (l + r) / 2;
26         mergeSort(a, l, m);
27         mergeSort(a, m + 1, r);
28         merge(a, l, m, r);
29     }
30 }
31
32 int main() {
33     int n;
34
35     printf("Enter size: ");
36     scanf("%d", &n);
37
38     if(n == 0) {
39         printf("List is empty\n");
40         return 0;
41     }
42 }

```

```

Enter size: 5
4 5 8 3 5
Maximum element: 8

=== Code Execution Successful ===

```

7. Remove Duplicate Elements

```
main.c
1 #include <stdio.h>
2
3 int main() {
4     int n, found;
5
6     printf("Enter size: ");
7     scanf("%d", &n);
8
9     int a[n], unique[n], count = 0;
10
11     for(int i = 0; i < n; i++)
12         scanf("%d", &a[i]);
13
14     for(int i = 0; i < n; i++) {
15         found = 0;
16
17         for(int j = 0; j < count; j++) {
18             if(a[i] == unique[j]) {
19                 found = 1;
20                 break;
21             }
22         }
23
24         if(!found)
25             unique[count++] = a[i];
26     }
27
28     printf("Unique elements: ");
29
30     for(int i = 0; i < count; i++)
31         printf("%d ", unique[i]);
32
33     printf("\n");
34
35     return 0;
36 }
```

Output

Enter size: 6
9 7 5 3 1 7
Unique elements: 9 7 5 3 1

=== Code Execution Successful ===

8. Bubble Sort

```
main.c
1 #include <stdio.h>
2
3 int main() {
4     int n, temp;
5
6     printf("Enter size: ");
7     scanf("%d", &n);
8
9     int a[n];
10
11     for(int i = 0; i < n; i++)
12         scanf("%d", &a[i]);
13
14     for(int i = 0; i < n - 1; i++) {
15         for(int j = 0; j < n - i - 1; j++) {
16             if(a[j] > a[j + 1]) {
17                 temp = a[j];
18                 a[j] = a[j + 1];
19                 a[j + 1] = temp;
20             }
21         }
22     }
23
24     printf("Sorted array: ");
25
26     for(int i = 0; i < n; i++)
27         printf("%d ", a[i]);
28
29     printf("\n");
30
31     return 0;
32 }
```

Output

Enter size: 5
8 4 2 0 9
Sorted array: 0 2 4 8 9

=== Code Execution Successful ===

9. Binary Search

main.c

Run

Clear

```

9   int a[n];
10
11  for(int i = 0; i < n; i++)
12      scanf("%d", &a[i]);
13
14  // Sort
15  for(int i = 0; i < n - 1; i++)
16      for(int j = 0; j < n - i - 1; j++)
17          if(a[j] > a[j + 1]) {
18              temp = a[j];
19              a[j] = a[j + 1];
20              a[j + 1] = temp;
21          }
22
23  printf("Enter key: ");
24  scanf("%d", &key);
25
26  low = 0;
27  high = n - 1;
28
29  while(low <= high) {
30      mid = (low + high) / 2;
31
32      if(a[mid] == key) {
33          found = mid;
34          break;
35      }
36      else if(a[mid] < key)
37          low = mid + 1;
38      else
39          high = mid - 1;
40  }
41
42  if(found != -1)
43      printf("Element %d is found at\n position %d\n", key, found + 1);
44  else
45      printf("Element %d is not found\n", key);
46
47  return 0;

```

line Java Compiler

Run

Clear

```

Enter size: 8
3 4 6 -9 10 8 9 30
Enter key: 10
Element 10 is found at position 7

=== Code Execution Successful ===

```

10. Sort Array Without Built-in Function- Merge Sort — $O(n \log n)$

main.c

Run

Clear

```

11  }
12
13  while(i <= m)
14      temp[k++] = a[i++];
15
16  while(j <= r)
17      temp[k++] = a[j++];
18
19  for(i = l, k = 0; i <= r; i++)
20      a[i] = temp[k++];
21  }
22
23  void mergeSort(int a[], int l, int r) {
24      if(l < r) {
25          int m = (l + r) / 2;
26
27          mergeSort(a, l, m);
28          mergeSort(a, m + 1, r);
29          merge(a, l, m, r);
30      }
31  }
32
33  int main() {
34      int n;
35
36      printf("Enter size: ");
37      scanf("%d", &n);
38
39      int a[n];
40
41      for(int i = 0; i < n; i++)
42          scanf("%d", &a[i]);
43
44      mergeSort(a, 0, n - 1);
45
46      printf("Sorted array: ");
47
48      for(int i = 0; i < n; i++)
49          printf("%d ", a[i]);
50
51      return 0;
52  }

```

line Java Compiler

Run

Clear

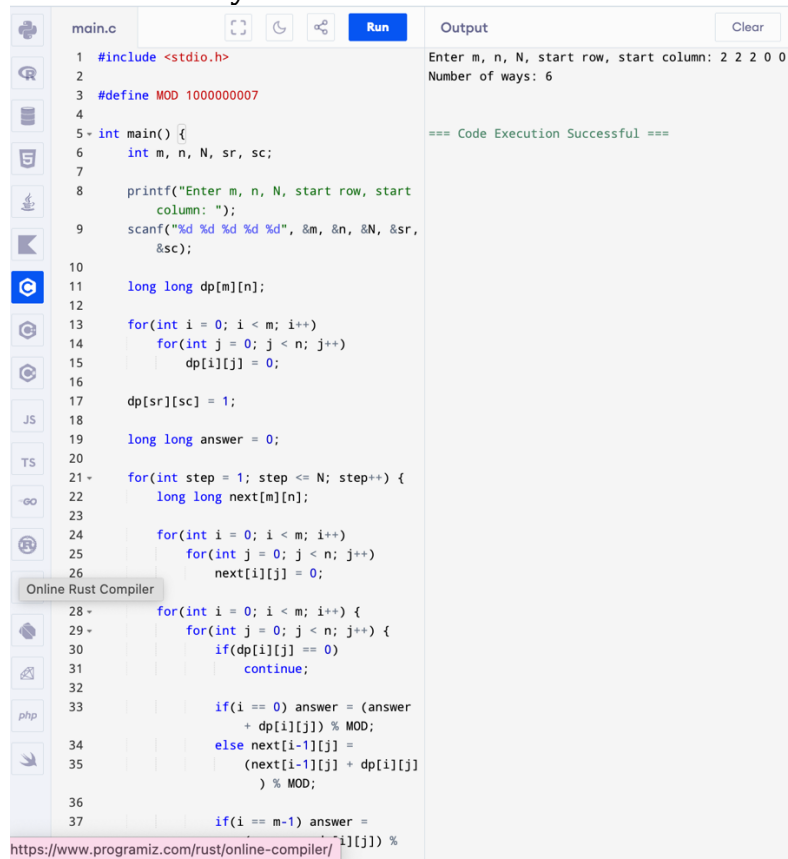
```

Enter size: 5
5 1 4 2 3
Sorted array: 1 2 3 4 5

=== Code Execution Successful ===

```

11. Number of Ways to Move Ball Out of Grid



The screenshot shows an online C++ compiler interface. The code is in a file named `main.c`. It includes `<stdio.h>` and defines a constant `MOD 1000000007`. The `main` function takes five integers as input: `m`, `n`, `N`, `sr`, and `sc`. It initializes a 2D array `dp` of size `m` by `n` with all values set to 0. It then sets `dp[sr][sc] = 1`. A variable `answer` is initialized to 0. A loop runs for `step` from 1 to `N`. In each iteration, a new 2D array `next` of size `m` by `n` is created, also initialized to 0. Another loop iterates over all cells `(i, j)` in the grid. If `dp[i][j] != 0`, it means there are ways to reach `(i, j)` in the previous step. For each of the four adjacent cells (up, down, left, right), if they are within the grid boundaries, the value of `next` at that cell is incremented by `dp[i][j]` modulo `MOD`. After processing all cells, the value of `answer` is updated to be the sum of all values in the `next` array, also modulo `MOD`. The final output is `answer`.

```
1 #include <stdio.h>
2
3 #define MOD 1000000007
4
5 int main() {
6     int m, n, N, sr, sc;
7
8     printf("Enter m, n, N, start row, start column: ");
9     scanf("%d %d %d %d %d", &m, &n, &N, &sr, &sc);
10
11     long long dp[m][n];
12
13     for(int i = 0; i < m; i++)
14         for(int j = 0; j < n; j++)
15             dp[i][j] = 0;
16
17     dp[sr][sc] = 1;
18
19     long long answer = 0;
20
21     for(int step = 1; step <= N; step++) {
22         long long next[m][n];
23
24         for(int i = 0; i < m; i++)
25             for(int j = 0; j < n; j++)
26                 next[i][j] = 0;
27
28         for(int i = 0; i < m; i++) {
29             for(int j = 0; j < n; j++) {
30                 if(dp[i][j] == 0)
31                     continue;
32
33                 if(i == 0) answer = (answer + dp[i][j]) % MOD;
34                 else next[i-1][j] = (next[i-1][j] + dp[i][j]) % MOD;
35
36                 if(i == m-1) answer =
37                     (answer + dp[i][j]) % MOD;
38                 else next[i+1][j] = (next[i+1][j] + dp[i][j]) % MOD;
39
40                 if(j == 0) answer = (answer + dp[i][j]) % MOD;
41                 else next[i][j-1] = (next[i][j-1] + dp[i][j]) % MOD;
42
43                 if(j == n-1) answer = (answer + dp[i][j]) % MOD;
44                 else next[i][j+1] = (next[i][j+1] + dp[i][j]) % MOD;
45             }
46         }
47
48         for(int i = 0; i < m; i++)
49             for(int j = 0; j < n; j++)
50                 dp[i][j] = next[i][j];
51     }
52
53     printf("Number of ways: %lld", answer);
54 }
```

Output:

```
Enter m, n, N, start row, start column: 2 2 2 0 0
Number of ways: 6

=== Code Execution Successful ===
```

12. House Robber II

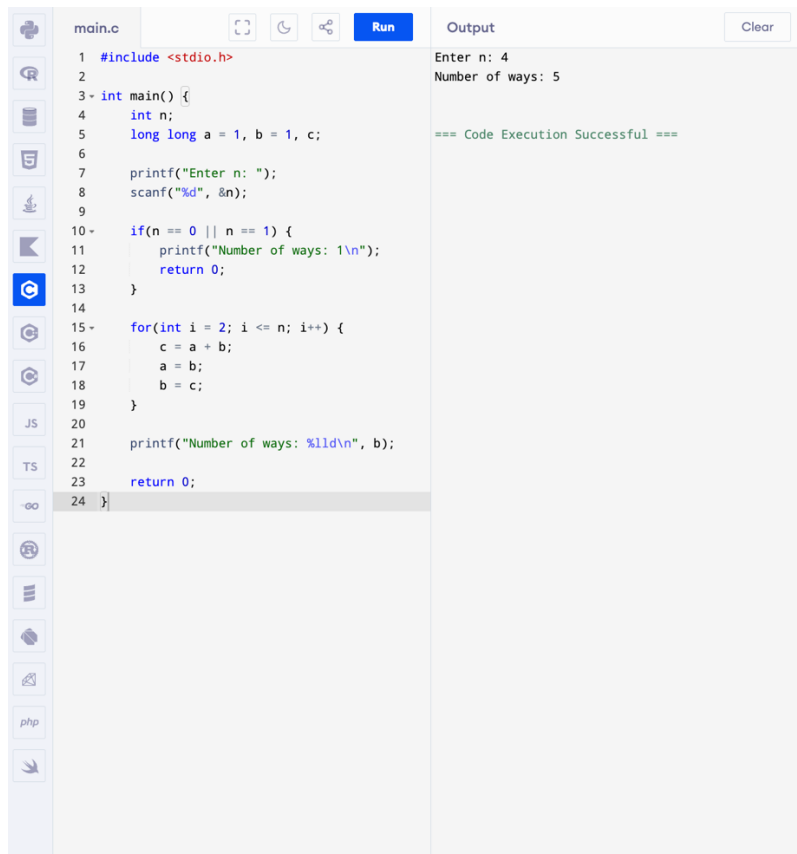
```
main.c
2
3 int max(int a, int b) {
4     return a > b ? a : b;
5 }
6
7 int robRange(int a[], int start, int end) {
8     int prev2 = 0, prev1 = 0;
9
10    for(int i = start; i <= end; i++) {
11        int current = max(prev1, prev2 + a[i]);
12        prev2 = prev1;
13        prev1 = current;
14    }
15
16    return prev1;
17 }
18
19 int main() {
20     int n;
21
22     printf("Enter number of houses: ");
23     scanf("%d", &n);
24
25     int a[n];
26
27     for(int i = 0; i < n; i++)
28         scanf("%d", &a[i]);
29
30     if(n == 1) {
31         printf("Maximum money: %d\n", a[0]);
32         return 0;
33     }
34
35     int result1 = robRange(a, 0, n - 2);
36     int result2 = robRange(a, 1, n - 1);
37
38     printf("Maximum money: %d\n", max(result1, result2));
39
40     return 0;
41 }
```

Output

Enter number of houses: 3
6 4 8
Maximum money: 8

=== Code Execution Successful ===

13. Climbing Stairs



```
main.c
1 #include <stdio.h>
2
3 int main() {
4     int n;
5     long long a = 1, b = 1, c;
6
7     printf("Enter n: ");
8     scanf("%d", &n);
9
10    if(n == 0 || n == 1) {
11        printf("Number of ways: 1\n");
12        return 0;
13    }
14
15    for(int i = 2; i <= n; i++) {
16        c = a + b;
17        a = b;
18        b = c;
19    }
20
21    printf("Number of ways: %lld\n", b);
22
23    return 0;
24 }
```

Output

Enter n: 4
Number of ways: 5

=== Code Execution Successful ===

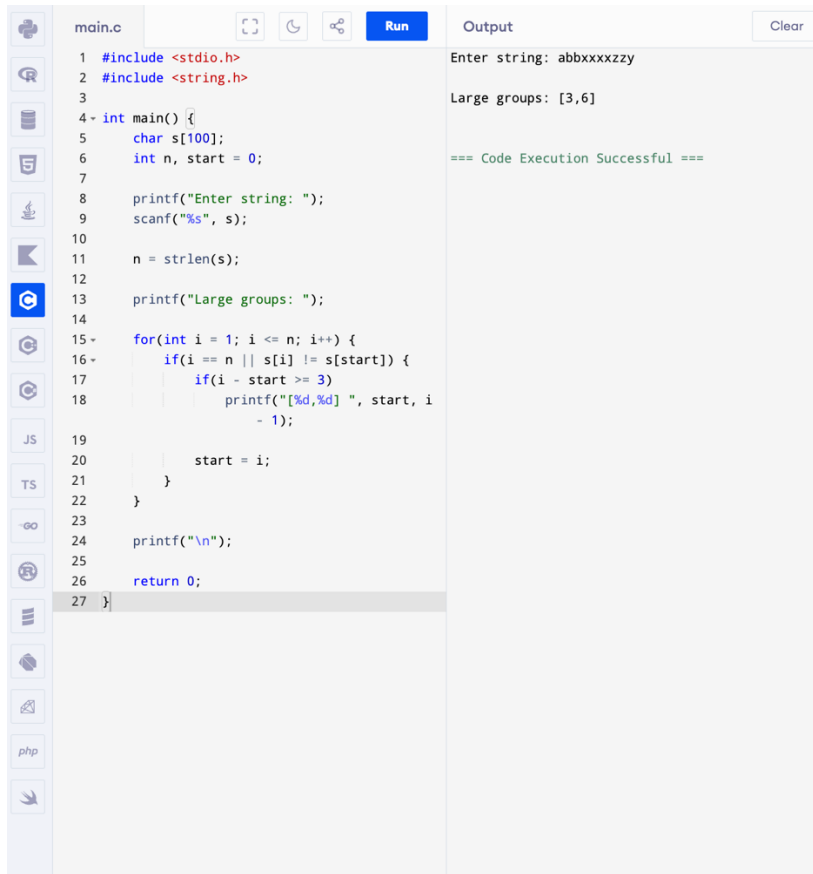
14. Unique Paths in Grid

The image shows a C++ IDE with a file named `main.c`. The code implements a dynamic programming algorithm to count the number of unique paths from the top-left to the bottom-right of a grid of size `m` by `n`. The grid is represented by a 2D array `dp` of type `long long`. The base case is `dp[0][j] = 1` for all `j`, representing the single path along the top edge. The algorithm then iterates over the grid, calculating the number of paths for each cell as the sum of paths from the cell above and the cell to the left. The final result is printed as `Number of unique paths: 28` for `m=7` and `n=3`. The output panel shows the execution was successful.

```
1 #include <stdio.h>
2
3 int main() {
4     int m, n;
5
6     printf("Enter m and n: ");
7     scanf("%d %d", &m, &n);
8
9     long long dp[n];
10
11     for(int j = 0; j < n; j++)
12         dp[j] = 1;
13
14     for(int i = 1; i < m; i++) {
15         for(int j = 1; j < n; j++) {
16             dp[j] = dp[j] + dp[j - 1];
17         }
18     }
19
20     printf("Number of unique paths: %lld\n",
21           dp[n - 1]);
22     return 0;
23 }
```

Output: Enter m and n: 7 3
Number of unique paths: 28
=== Code Execution Successful ===

15. Large Groups in String



The image shows a code editor with a sidebar on the left containing icons for various programming languages: C, C++, Java, JavaScript, TypeScript, Python, PHP, and others. The main editor area is titled 'main.c' and contains the following C code:

```
1 #include <stdio.h>
2 #include <string.h>
3
4 int main() {
5     char s[100];
6     int n, start = 0;
7
8     printf("Enter string: ");
9     scanf("%s", s);
10
11     n = strlen(s);
12
13     printf("Large groups: ");
14
15     for(int i = 1; i <= n; i++) {
16         if(i == n || s[i] != s[start]) {
17             if(i - start >= 3)
18                 printf("[%d,%d] ", start, i - 1);
19
20             start = i;
21         }
22     }
23
24     printf("\n");
25
26     return 0;
27 }
```

On the right side of the editor, there is an 'Output' panel with a 'Clear' button. It displays the following text:

```
Enter string: abxxxxzy
Large groups: [3,6]
=== Code Execution Successful ===
```

16. Game of Life

```
main.c Run Output Clear
9 int board[m][n], next[m][n];
10
11 for(int i = 0; i < m; i++)
12     for(int j = 0; j < n; j++)
13         scanf("%d", &board[i][j]);
14
15 int dr[] = {-1,-1,-1,0,0,1,1,1};
16 int dc[] = {-1,0,1,-1,1,-1,0,1};
17
18 for(int i = 0; i < m; i++) {
19     for(int j = 0; j < n; j++) {
20         int live = 0;
21
22         for(int k = 0; k < 8; k++) {
23             int r = i + dr[k];
24             int c = j + dc[k];
25
26             if(r >= 0 && r < m && c >= 0
27                && c < n)
28                 live += board[r][c];
29
30             if(board[i][j] == 1)
31                 next[i][j] = (live == 2 ||
32                    live == 3) ? 1 : 0;
33             else
34                 next[i][j] = (live == 3) ? 1
35                    : 0;
36         }
37     }
38     printf("Next generation:\n");
39     for(int i = 0; i < m; i++) {
40         for(int j = 0; j < n; j++)
41             printf("%d ", next[i][j]);
42         printf("\n");
43     }
44 }
45
46 return 0;
47 }
```

Enter rows and columns: 4 3
0 1 0
0 0 1
1 1 1
0 0 0

Next generation:
0 0 0
1 0 1
0 1 1
0 1 0

=== Code Execution Successful ===

17. Champagne Tower

